Quy gán nguồn cho RAG hai tầng

Shapley ở tầng sinh · Integrated Gradients trên đồ thị ở tầng truy hồi

Câu hỏi: trong các đoạn được truy hồi, đoạn nào thật sự tạo ra câu trả lời —
và đáng lẽ retriever phải đi lối nào trong đồ thị để tới các đoạn đó?

Tài liệu thiết kế · chưa có kết quả thực nghiệm

Bài toán

Mô hình trả lời xong. Ta đưa cho nó tới 60 đoạn truy hồi. Đoạn nào thật sự tạo ra câu trả lời đó?



Khó ở chỗ thứ nhất — dư thừa

Ba đoạn cùng nói một ý thì không thể tính công đầy đủ cho cả ba. Như vậy là cộng khống.

Và bỏ riêng từng đoạn thì mất rất ít — nên phép thử «bỏ một đoạn» cũng không phát hiện được.

Khó ở chỗ thứ hai — chi phí

Cách công bằng về mặt toán học là giá trị Shapley: thử mọi tổ hợp đoạn.

Với K = 60 đoạn, đó là $2^{60} \approx 10^{18}$ lượt chạy mô hình. Không khả thi.

Chi tiết: tài liệu thiết kế §1, §3.1

Nền: lý thuyết trò chơi hợp tác, trong một ví dụ

Cầu nối: đoạn văn là người chơi

```
người chơi      một đoạn văn trong pool
liên minh S     một tập đoạn  (có 2^n tập)
hàm đặc trưng   v(S) = chất lượng câu trả lời
                       khi chỉ đưa S cho model
```

Đặt ngưỡng → trò chơi thắng / thua

```
S thắng   <=>   v(S) >= tau
                "chừng này đoạn là đủ"

liên minh thắng tối tiểu  (MWC)
   thắng, mà bỏ ai cũng thành thua
```

Ví dụ ba đoạn a, b, c

```
v(rỗng) thua   v(a) thua   v(b) thua   v(c) thua
v(ab) thắng    v(ac) thắng
v(bc) thua     v(abc) thắng
MWC  =  { a,b }  và  { a,c }
{a,b,c} thắng nhưng không tối tiểu
– bỏ c vẫn thắng
```

Ba chỉ số, cùng một kết luận: a là then chốt

```
PGI         tỉ lệ MWC có chứa i
            a: 1       b: 1/2     c: 1/2
Deegan-P    chia đều trong mỗi MWC
            a: 1/2     b: 1/4     c: 1/4
Banzhaf     tỉ lệ swing (thua -> thắng)
            a: 3/4     b: 1/4     c: 1/4
```

Shapley — và vì sao tài liệu này không dùng nó

φ_i = đóng góp biên trung bình của i trên mọi thứ tự xuất hiện. $\Sigma\ \varphi_i = v(\text{toàn bộ}) - v(\emptyset)$ ← tiên đề đầy đủ

Công bằng theo tiên đề, nhưng cần duyệt 2^n liên minh — với n = 40 là bất khả thi.

Và câu ta cần không phải chia mà là chọn: tập nào là đủ. Bản Shapley giữ ở tài liệu riêng.

trội Pareto — A tốt hơn hoặc bằng B ở mọi tiêu chí thì B bị loại. đơn điệu — thêm người chơi không làm thắng thành thua; ta không có.

Chi tiết: §2.1

Toàn cảnh — hai tầng, hai câu hỏi khác nhau

Tầng A hỏi

Tập nào đủ và tối tiểu?

Tầng B hỏi

Đáng lẽ phải đi lối nào tới đó?



Phần đắt nằm hết ở offline. Lúc chạy thật: truy vấn → GFM đã train → đoạn → LLM → trả lời.

Chi tiết: §1, sơ đồ luồng

Tầng A · pha 1 — bốn vòng thu thập dữ liệu

g_θ chỉ cần trả lời một câu: «g_θ(S) ≥ tau_q ?». Nên nó là bộ phân loại ở biên, không phải bộ hồi quy — và lấy mẫu phải dồn vào biên. Cặp đôi ngẫu nhiên hay tập con đều rơi xa ngưỡng: negative dễ, vô dụng.

vòng 0	**khung bắt buộc**	v(∅), v(C), v(R), mọi singleton — rẻ nhất (750 token), bắt ca «một đoạn tự đủ»	44.500
vòng 1	**đi dọc biên**	greedy từ rỗng, ghi cả ứng viên bị loại — đường cắt ngang tau_q một lần	27.000
vòng 2	**active learning**	nơi ensemble bất đồng và dự đoán rơi quanh tau_q	37.500
vòng 3	**xác nhận**	S* và top-k bằng model thật — không để proxy quyết định nhận/bỏ	18.750

r là kết quả, không phải núm vặn

Greedy thêm một đoạn mỗi vòng tới khi vượt tau_q,
nên r = |S| lúc dừng = cỡ tập đủ tìm được.
2-hop → r = 2. r = 3 chỉ là ước lượng tính tiền.

Vòng greedy ĐẦU TIÊN miễn phí — nó chính là chấm singleton. Chi phí = (r − 1)·m, không phải r·m.

m thích nghi — prior mù đúng ở ca ta cần

multi-hop: MỌI singleton ≈ v(∅) → prior là nhiễu.
Chọn 12 trên 40 ≈ bốc ngẫu nhiên (29% trúng).

spread = max v({k}) − v(∅) đã đo sẵn ở vòng 0
≥ 0.15·cái bánh → m = 12 (prior tốt)
< ngưỡng → m = 30 (+32% token)

Điểm chốt

m nhỏ chưa chắc rẻ: shortlist trượt thì r tăng — chi phí chuyển từ «vòng rộng» sang «nhiều vòng», mà S* lại kém hơn.

Chi tiết: §3.4

Tầng A · từ MWC tới đầu ra

1 · Liệt kê

Beam search trên g_θ, 0 lượt LLM.

Đừng dừng ở cỡ đầu tiên đạt đủ — chạy tiếp 1–2 cỡ nữa.

Mở rộng trong cụm để lấy lại PGI mà shortlist đã giấu mất.

2 · Chỉ số cấu trúc

Banzhaf có dấu:

β^+ thiếu → đủ

β^- đủ → thiếu

$\beta < 0$ chỉ thẳng ra đoạn gây hại — negative khó cho V_neg.

Chuẩn hóa β và PGI riêng rồi gộp.

3 · Chọn S*

parsimony $-|S|$

structure σ_g

rank Σr_i

Lọc Pareto, rồi gộp có trọng số.

Rerank chỉ phá hòa ở bước cuối.

Đầu ra — và chỉ dòng đầu đi qua ranh giới hai tầng

T = S*	tập đích, sang tầng B	← không điểm số nào đi qua
W, p(S)	họ lời giải thích thay thế	
entropy(p)	độ tin cậy — cao nghĩa là nhiều MWC suýt soát, quy gán không chắc	
$\beta_i < 0$	negative khó cho L_node (§6.3)	

Chi tiết: §3.6 – §3.10, §8

Tầng A · công thức — v, τ, và tập đủ tối tiểu

Hàm giá trị

```
a^_S    =  generate( q, S )
v(S)    =  P_judge( "đúng" | q, a^_S, y )  ∈ [0,1]
           đọc logprob token đầu, không decode
v(rỗng) =  điểm chỉ bằng tri thức tham số
```

Ngưỡng đủ

```
tau_q    =  theta × [ v_ref(q) − v(rỗng) ] + v(rỗng)
            theta = 0.95
v_ref(q) =  max( v(C),  max v(S) đã quan sát )
chốt chặn:  v_ref − v(rỗng) < 0.05  →  T = rỗng
```

Định nghĩa

```
S đủ          v(S) ≥ tau_q  VÀ  adq(S) = phủ hết
S tối tiểu    đủ,  và với mọi c thuộc S:
                 v( S \ {c} )  <  tau_q
W  =  { mọi S đủ tối tiểu }
```

Bổ đề: không cần v đơn điệu

```
ṽ(S) = max v(S')  trên mọi S' con S     bao đơn điệu
S thắng tối tiểu của ṽ  ⟹  thắng nhờ một S' con S
   S' thật sự con S  ⟹  S' cũng thắng, trái tối tiểu
   ⟹  S' = S,  tức v(S) ≥ tau_q
MWC( ṽ ) = W      đơn điệu hóa không đổi gì
```

Điểm chốt

«Đủ» có HAI thành phần — xem slide sau. Và lý thuyết simple game đòi đơn điệu, định nghĩa trên thì không.

Chi tiết: §3.1 – §3.3

Tầng A · «đủ» có hai thành phần, và v chỉ đo một

v đo «trả lời đúng», không đo «có bằng chứng»

```
v(S)    model đọc S rồi trả lời — khớp y không?
adq(S)  các đoạn trong S có THẬT SỰ nói ra y?

q = "ai đạo diễn phim thắng Oscar 1994?"  y = Zemeckis
đoạn 19 = "Forrest Gump là hiện tượng văn hóa 1990s"
   không nói đạo diễn, không nói thắng giải
   model thấy tên phim → TỰ NHỚ → trả lời ĐÚNG
   v({19}) = 0,71 cao → bản cũ nhận 19 vào T
   → rồi dạy retriever đi lấy nó.  NHÃN SAI.
```

Và lệch này đã được đo, không phải giả định

```
HotpotQA  model trả lời ĐÚNG 35–62% số ca mà
          ngữ cảnh KHÔNG đủ — nhờ nhớ sẵn

tỷ lệ trả lời sai   10,2%  không ngữ cảnh
                 →  66,1%  ngữ cảnh không đủ

tỷ lệ ca CÓ ngữ cảnh đủ sau truy hồi:
   MuSiQue-Ans 46,2%    ·    HotpotQA 44,6%
→ hơn NỬA truy vấn trên đúng hai bộ ta nhắm
  KHÔNG có tập đủ trong pool
```

Vì sao giữ TÁCH khỏi v, không gộp thành một điểm số

	adq phủ hết	adq KHÔNG phủ
v ≥ τ_q	nhãn TỐT — nhận	MỒI THAM SỐ — loại ← lỗ bản cũ không thấy
v < τ_q	bằng chứng CÓ mà model không nối được → lỗi MODEL SINH, tầng B KHÔNG sửa được	pool THIẾU — thoát sớm ở A0/C0 → cờ «pool thiếu», nuôi ô «lỗi đồ thị» §6.1

Chi tiết: §3.2 · Joren et al. ICLR 2025, arXiv:2411.06037

Tầng A · adq đo thế nào — chẻ câu hỏi và bảng phủ

Hai bước, hai nơi hỏi

```
bước 1  CHẺ câu hỏi thành mảnh
   hỏi ai:  MuSiQue gán sẵn (phân rã 1-hop +
            đáp án trung gian), hoặc LLM
            1 lượt MỖI TRUY VẤN — không mỗi tập con
   mảnh 1 "phim thắng 1994 là Forrest Gump"
   mảnh 2 "Forrest Gump do Zemeckis đạo diễn"

bước 2  ĐỐI CHIẾU mỗi đoạn × mỗi mảnh
   hỏi ai:  model NLI nhỏ ~400M, local, 0 LƯỢT API
   ra:      entail / neutral / contradict
```

Bảng phủ — 0 lượt LLM, cho ba thứ

	mảnh 1	mảnh 2		
đoạn 3	entail	—	1	CỬA THOÁT
đoạn 7	entail	—		mảnh nào trống
đoạn 12	—	entail		→ bỏ truy vấn ngay
đoạn 23	—	entail	2	ỨNG VIÊN
đoạn 31	—	entail		phủ tập, giải
đoạn 19	—	—		CHÍNH XÁC
			3	PRIOR |T| = số mảnh

adq(S) = phủ hết ⇔ mọi mảnh có đoạn trong S entail
nghiệm phủ: {3,12} {3,23} {3,31} {7,12} {7,23} {7,31}

Vì sao PHẢI chẻ ra — hai lý do độc lập, cả hai đều chặn

ĐỘ DÀI premise = cả S có thể tới 10.000 token, mà model NLI chỉ nhận 512. Chẻ ra thì mỗi lượt kiểm là 1 đoạn (250 token) đối 1 câu ngắn — vừa thoải mái.

BẮC CẦU claim ghép «phim thắng 1994 do Zemeckis đạo diễn» thì KHÔNG đoạn nào một mình khẳng định được → loại oan cả {3,12}, tức loại đúng ca multi-hop. Chẻ ra thì mỗi mảnh có đúng một đoạn khẳng định nó. (arXiv:2608.00585 — lấy cơ chế, chưa trích số)

Chi tiết: §3.2, §3.11 · arXiv:2608.00585

Tầng A · surrogate g_θ — một hàm, hai pha

Một hàm duy nhất, cho cả hai pha

```
g_θ( q, S, đặc trưng mỗi chunk ) → dự đoán v(S)
bất biến hoán vị trên S, vì S là tập
q          embed truy vấn      v(∅)  mốc dưới thang
mỗi k ∈ S  embed chunk
   + ŝ(q,k)  điểm rerank     LUÔN có, cả hai pha
   + ṽ_k     v({k}) − v(∅)   chỉ khi có vòng 0
KHÔNG đầu vào:   τ_q ,  v_ref
```

Dữ liệu — log của chính các lượt gọi v

```
pha 1, bốn vòng      10.000 × 111 = 1.200.000
verification pha 2  150.000 ×   7 = 1.050.000
leo thang            20.000 × 111 = 2.220.000
thoát sớm            20.000 ×   2 =    40.000
                                   ─────────
                                     4.510.000
|S| từ 0 tới 40 — nên MỘT mô hình, không phải hai
```

Che ṽ_k theo từng chunk — ba khuôn

```
pha 1 + leo thang    cả 40 chunk có ṽ_k
verification pha 2   CHỈ |S*| chunk có
beam ở bước C5       không chunk nào có

Trộn ba khuôn theo đúng tỷ lệ xuất hiện. Che
tất-cả-hoặc-không-gì không dạy được khuôn giữa
— mà đó là khuôn của MỌI lượt pha 2.
```

Nghiệm thu, và sàn phải vượt

```
AUC TRONG từng truy vấn, rồi trung bình
sàn:  mô hình chỉ nhìn |S|

AUC gộp là bẫy — tập lớn thì v cao nên số đẹp
sẵn, mà phân biệt trong truy vấn bằng 0.
Mốc kiến trúc: GBM đặc trưng tay chạy TRƯỚC.
Set Transformer là ĐỀ XUẤT, chỉ chọn nếu vượt nó.
```

Điểm chốt — phần phải học là độ lệch khỏi cộng tính

```
Hai đoạn trùng ý → lệch ÂM.  Hai đoạn bắc cầu → lệch DƯƠNG.  Đó là quy luật chung, nên gộp nhiều truy vấn
có nghĩa. Còn τ_q thì KHÔNG phải đầu vào: bước C2 gọi g_θ để quyết định có mua v(C) — mà v(C) mới chốt τ_q.
```

Chi tiết: §3.5

Tầng A · hai pha — mô hình ở pha 1, nhãn ở pha 2

Vì sao phải tách

```
chạy pipeline đầy đủ cho 200.000 truy vấn
    200.000 × $0,38  =  $76.000   không trả nổi
PHA 1  10.000 truy vấn, LLM đầy đủ
       sản phẩm: g_θ — KHÔNG phải nhãn
PHA 2  200.000 truy vấn, đường rẻ
       sản phẩm: nhãn (q, T) cho tầng B
```

Giữa hai pha — chỉ ba bước

```
B1  gộp log            ~1.110.000 mẫu
B2  train g_θ          che ṽ_k ba khuôn
B3  CỔNG p — giữ 500 truy vấn pha 1, chạy cả
    đường pha 2, so với S* thật đã biết
    0 lượt LLM. Chạy TRƯỚC khi cam kết N₂.
    p ≥ 0,90 chạy · p ≈ 0,75 tăng N₁ trước
```

Đường pha 2 — ~22.300 token nếu chạy đủ; kỳ vọng 12.400 ≈ $0,037 vì ~45% thoát ở C0

```
C0  chẻ câu hỏi + bảng phủ NLI (§3.2) — thiếu mảnh → THOÁT, cờ «pool thiếu»          400 token
C1  v(∅)                                                                          500 token
C2  g_θ chấm 40 chunk đơn, ṽ_k che                                                  0 token
C3  v(R)    R = 12 chunk — SÀN CỤM thắng hạn ngạch nguồn: 6 suất mỗi cụm 1, rồi 6 suất   3.500 token
            bảo đảm mỗi nguồn ≥ 1 trước khi lấp theo hạng
C4  v(C)    cả 40 chunk — mua cho MỌI truy vấn, không bù bằng hằng số          10.500 token
      ──  v_ref = max( v(R), v(C) ) ;   τ_q chốt tại đây  ──
      v_ref − v(∅) < 0,05  →  T = ∅, cờ «truy hồi không đóng góp»  (chốt chặn §3.3)
C5  TÌM THEO TẦNG từ nghiệm phủ của C0 (§3.6) — beam mò làm dự bị                   0 token
C6  verification bằng LLM THẬT — đủ, và tối tiểu                                ~7.400 token
C7  ba nhánh: xuất nhãn  ·  thử ứng viên kế  ·  cứu
```

Chi tiết: §3.11

Tầng A · ba ngưỡng |S*|, và ba nhánh của C7

Quét hết nổ theo 2^s — tỉa, hoặc dùng bảng phủ

```
S ← S* ứng viên
lặp khi |S| > 5:
    xếp c ∈ S theo g_θ(q, S\{c}) giảm dần   0 lượt
    thử c đầu bảng: đo v(S\{c})
        ≥ τ_q → S ← S\{c}, lặp
        < τ_q → thử c kế, tối đa 3 lần
    3 lần đều trượt → KẸT
|S| ≤ 5 → quét hết dàn dưới S → T, CHỨNG MINH
```

Tỉa rồi quét VẪN là chứng minh đầy đủ

§3.10 chỉ đòi T LÀ MỘT MWC — không đòi duy nhất hay nhỏ nhất.
T = {a,b,c} với v(T) ≥ τ_q và cả 6 tập con thực sự < τ_q ⇒ T là tập đủ tối tiểu. 247 tập con khác của một S* cỡ 8 không cần xét.
CÓ bảng phủ thì rẻ hơn nữa: chỉ tập con CÒN PHỦ mới đe dọa tối tiểu — T={3,12,19} chỉ còn {3,12}, mà tầng 0 đã đo. Ba ngưỡng dưới chỉ cần cho nhánh beam mò dự bị.
Cái không đầy đủ là W — mà W ở pha 2 vốn đã không dùng để kết luận cấu trúc.

Ba ngưỡng, và ba nhánh

|S*|	2	3	5	6	8	10	15	20	
quét hết	$0,009	$0,020	$0,107	$0,239	$1,14	$5,37	$172	$5.500	nổ theo 2^s
tỉa+quét	—	—	$0,107	$0,043	$0,071	$0,098	$0,218	$0,326	tuyến tính theo s
cứu	$0,34	$0,34	$0,34	$0,34	$0,34	$0,34	$0,34	$0,34	

|S*| ≤ 5 quét hết ngay · 6 ≤ |S*| ≤ 10 tỉa xuống ≤ 5 rồi quét hết · |S*| ≥ 11 cứu
tỉa còn rẻ hơn cứu tới s ≈ 20, nên ranh giới 10/11 KHÔNG đặt theo tiền — ở cỡ đó ta không còn tin điểm xuất phát đủ để tỉa 15 lượt theo chỉ dẫn của chính mô hình đang sai. Tỉa KẸT ở ≥ 6 → cứu.

C7 (1) đủ + tối tiểu → T sang tầng B (2) v(S*) < τ_q → thử ứng viên kế, tối đa 3 lần, mỗi lần CHỈ 1 lượt (1.250 token; một lượt cứu = 113.000, mua được 90 lần thử) → p 0,75 → 0,91
(3) |S*| ≥ 11 · tỉa kẹt · 3 lần thử đều trượt → hàng cứu có phân tầng

Chi tiết: §3.10, §3.11

Tầng A · chỉ số cấu trúc, chọn S*, phát credit

Chỉ số trên từng đoạn

```
Banzhaf có dấu:
   beta⁺  số lần thêm i:  thiếu → đủ
   beta⁻  số lần thêm i:  đủ → thiếu
   beta_i = ( beta⁺ − beta⁻ ) / 2^(n−1)
PGI_i      = | { S thuộc W : i thuộc S } |
overlap(S) = Σ trên S'≠S  |S ∩ S'| / |S ∪ S'|
```

Gộp thành chỉ số trên từng MWC

```
chuẩn hóa beta và PGI riêng rồi mới gộp
   beta trong [−1,1],  PGI là số đếm
   cộng thẳng thì PGI nuốt beta

sigma_g^sum(S) = Σ trên i∈S  ( beta~_i + PGI~_i )
sigma_g^min(S) = min trên i∈S ( beta~_i + PGI~_i )
```

Chọn S*

```
sigma_s(S) = −|S|                    gọn
sigma_g(S)                           cấu trúc
sigma_r(S) = Σ trên i∈S  r_i         khớp rerank

S* = argmax  w_s·s~_s + w_g·s~_g + w_r·s~_r
lọc Pareto trước; rerank chỉ phá hòa cuối
```

Phát credit

```
cứng  credit(c_i) = 1/|S*| nếu c_i thuộc S*

mềm   p(S) = softmax( Phi(S) / T )
      credit(c_i) = Σ trên S chứa i  p(S)/|S|
      T → 0        quay về bản cứng
      T → vô cùng  = Deegan−Packel chuẩn
```

Điểm chốt

entropy(p) là tín hiệu độ tin cậy: cao nghĩa là nhiều MWC suýt soát, quy gán không chắc.

Chi tiết: §3.7 – §3.9

Integrated Gradients — công cụ chính

Câu hỏi IG trả lời: một đầu ra vô hướng F chịu ảnh hưởng bao nhiêu từ mỗi đầu vào?

$$IG_i = (x_i - b_i) \times \int \text{gradient dọc đường từ mốc b tới điểm thật x}$$

đi xa mốc bao nhiêu — không chỉ lấy gradient tại đích

Vì sao phải tích phân — ví dụ

F(a,b) = min(a+b, 1), mốc (0,0), thật (0.8, 0.6)

gradient tại điểm thật: a+b = 1.4 > 1 → bão hòa
∂F/∂a = ∂F/∂b = 0
«cả hai đều vô dụng» — sai hẳn

IG dọc đường: IG_a = 0.57, IG_b = 0.43

Tính chất dùng xuyên suốt — completeness

$\Sigma IG_i = F(x) - F(\text{mốc})$

Trong ví dụ: 0.57 + 0.43 = 1.00 ✓

Các phần cộng lại đúng bằng cái tổng. Nhờ nó mọi con số IG đọc được như «chiếm bao nhiêu phần trăm», và mọi phép kiểm đều dựa vào nó.

Hai nghĩa của chữ «đường» — đừng lẫn

đường tích phân — đoạn thẳng trong không gian đầu vào, từ mốc tới điểm thật — thiết bị toán học bên trong IG
đường trên đồ thị — chuỗi cạnh seed → … → D — thứ tầng B đi tìm

Chi tiết: §4 mở đầu

G-reasoner — kiến trúc, và ba chỗ quan trọng với ta

QuadGraph — bốn loại node, nối nhau hai chiều

community cụm chủ đề, có bản tóm tắt
document chunk văn bản
entity thực thể
attribute thuộc tính

Cạnh liên tầng cho phép entity → document → entity
khác: multi-hop ở ngay mức tài liệu.

Một lượt chạy

h^0_v = MLP(emb(v), emb(q)) nếu v ∈ V_q
= emb(v) nếu không

L tầng, L = 3–6:
m_e = DistMult($h_{v'}$, h_r, h_v)
h_v = Upd(h_v, Σ m_e)

$p(v)$ = $\text{Predictor}_{\text{type}(v)}$(h^L_v, emb(v), emb(q))

Ba tính chất — hai trong số đó là lý do ta phải can thiệp

inductive	mọi đầu vào suy từ văn bản, không tham số gắn ID node	chạy được trên đồ thị mới, không train lại ✓
không có trọng số cạnh	DistMult rồi cộng tổng — không attention, không hệ số riêng từng cạnh	IG không có gì để bám → chèn gate, §4.2
truy vấn chỉ vào lúc khởi tạo	sau đó L tầng lan truyền chạy như nhau bất kể hỏi gì	gate phụ thuộc truy vấn, §4.2 chế độ 2

34M tham số · EM 81,1 trên HotpotQA · arXiv:2509.24276 — chi tiết kiến trúc nên đối chiếu bản gốc

Tầng B · bối cảnh và chỗ thiếu

Graph foundation model — và ta dùng loại node nào

G-reasoner — QuadGraph bốn loại node. 34M tham số, không nhớ thực thể nào nên chạy được trên đồ thị mới.

document · community vào prompt ← nhãn T
entity · attribute không ← nhãn Π

Quy tắc: vào prompt ⟺ từng là người chơi của Shapley. Hai loại sau là chất nền lan truyền, không phải văn bản.

Vì sao phải là đồ thị

Bi-encoder chấm từng đoạn độc lập, nên không diễn đạt được «d chỉ có ích khi đi cùng c».

A: X sinh ở thành phố C v({A}) ≈ 0
B: C thuộc nước D v({B}) ≈ 0
hỏi «X sinh ở nước nào?» v({A,B}) cao

Đồ thị biểu diễn tính kết hợp một cách tự nhiên.

Chỗ thiếu khiến ta phải can thiệp

G-reasoner tính thông điệp bằng DistMult rồi cộng tổng: không attention, không hệ số riêng cho từng cạnh — mỗi cạnh hoặc gửi đủ, hoặc không tồn tại, không có mức trung gian.

Nhưng IG cần một biến liên tục vô hướng mới lấy đạo hàm được. Không có biến nào thì IG không bám vào đâu.

G-reasoner: arXiv:2509.24276 · GFM-RAG: arXiv:2502.01113, NeurIPS 2025

Tầng B · chèn gate lên cạnh

thông điệp mới = g × Msg(·)

g là một số trong [0,1] Msg(·) là vector d chiều

Một số duy nhất điều khiển cả vector — nhờ đó mỗi cạnh chỉ có một đại lượng cần quy gán, không phải d đại lượng.

Chế độ 1 — cách đo

g = 1 cho mọi cạnh, mọi tầng.

Nhân với 1 nên model chạy y hệt bản gốc.
Gate chỉ tồn tại để IG có chỗ bám.

Dùng để đo ở mọi mốc — không bao giờ bỏ.

Chế độ 2 — một tính năng của model

$\hat{g} = \sigma(\text{MLP}(h_v , h_r , h_{v'} , h_q))$

Cho model định tuyến khác nhau theo từng câu hỏi — bản gốc chỉ cho truy vấn tác động lúc khởi tạo. Bật từ mốc 2. Vẫn inductive.

Hai chế độ không phải hai lựa chọn cho cùng một việc

$\hat{g}$ là thứ model làm, không phải thứ ta dùng để đo. Nếu đo bằng $\hat{g}$ thì cạnh model đang đóng nhận IG ≈ 0 — mà đó đúng là những cạnh «lẽ ra phải mở» cần tìm. Nên đào nhãn thì luôn đo với g = 1, kể cả từ mốc 2.

Chi tiết: §4.2, §4.4

Tầng B · gate phải riêng từng tầng

Một cạnh vật lý được dùng L lần, mỗi tầng một lần — và các lần đó không tương đương.



Tầng 1

X → C nổ	C nhận tín hiệu truy vấn	hữu ích
C → D nổ	C còn trống, D nhận đồ vô nghĩa	vô dụng

Tầng 2

C → D nổ	C đã có tín hiệu, D nhận được	hữu ích
X → C nổ	C đã có rồi, thừa	gần vô dụng

Gate chung cho cả L tầng sẽ trộn hai vai trò đó lại và mất thông tin thứ tự — tức mất đúng cái quan trọng nhất của multi-hop. Nên số gate là $|\mathcal{E}| \times L$, một gate cho mỗi cặp (cạnh, tầng).

Điểm chốt

Cùng một cạnh: tầng này thiết yếu, tầng kia vô dụng. Thứ tự chính là nội dung của multi-hop.

Chi tiết: §4.3

Tầng B · công thức — gate, F_B, và IG

Gate chèn vào thông điệp

```
m_e^(l) = g_e^(l) * Msg( h_v^(l-1), h_r^(l), h_v'^(l-1) )
          ^vo huong      ^vector d chieu

h_v^(l) = Upd( h_v^(l-1), tong_{e vao v} m_e^(l) )

che do 2:  g^ = sigmoid( MLP(h_v,h_r,h_v',h_q) )
```

Điểm mục tiêu

```
p(v)   = Predictor_{type(v)}( h_v^(L), emb(v), emb(q) )

F_B(g) = log p(v_D)        riêng từng đích D

F_B là hàm của vector gate, không phải của
embedding -- khác biệt cốt lõi số với tầng A.
```

IG trên gate

```
IG_e^(l) = ( g*_e^(l) - 0 ) x (1/M) tổng_{m=1..M}  dF_B( alpha_m * g* ) / dg_e^(l)
           ^đầu vào - baseline        ^gradient trung bình doc đường

g* = 1   khi đào nhãn   -- thừa số đầu rụng hết, con dung gradient trung bình
   = g^  khi báo cáo    -- cạnh model da tắt (g^ ~ 0) tự động nhận IG ~ 0

Kiểm tra 4:   tổng_{e,l} IG_e^(l) = F_B(g*) - F_B(0)        completeness
```

Điểm chốt

ĝ là thứ model làm, không phải thứ ta dùng để đo. Đào nhãn thì luôn g* = 1.

Chi tiết: §4.2 – §4.4

Tầng B · tính chất c/h — dẫn xuất

Một đường chỉ truyền được tin khi mọi gate trên nó đều mở — đóng một gate ở giữa là cả đường vô dụng.

Đóng góp của đường là tích, không phải tổng

số hạng của đường = c * $\text{tich}_{i=1..h}$ (g_i / g*_i)

tai g = g* mọi thừa số = 1 -> số hạng = c (chuẩn hóa)
IG vấn g = alpha*g* mọi thừa số = alpha -> số hạng = c * alpha^h
đường càng dài càng tắt nhanh khi alpha giam

IG_i = g*_i * [tich phần tử 0 den 1] (c * alpha^(h-1) / g*_i) d(alpha)
= c * [alpha^h / h] từ 0 đến 1
= c / h

Vì sao kết quả sạch như vậy

g*_i triệt tiêu -> không phụ thuộc gate đó mở
to hay nhỏ, cung không phụ thuộc điểm cuối

Đối xứng: mọi hop đều thiết yếu như nhau
Completeness: tổng phải bằng c
-> hai điều đó khóa chặt đáp án

Dùng làm phép kiểm miễn phí

kiểm tra 5
đường h hop, tổng giá trị c
=> mỗi hop nhận c/h

Lệch nhiều -> đang nhìn hai đường chồng nhau
chu không phải một đường

Chi tiết: §4.5

Tầng B · thứ cần đọc ra là một chuỗi

IG cho mỗi cặp (cạnh, tầng) một con số. Đó là một đống số rời rạc — chưa phải lời giải thích.

Ví dụ: «vợ của đạo diễn phim X là ai?»



Vì sao không lấy top-k cạnh điểm cao ✗

X --0.18--> A P --0.21--> Q R --0.19--> D
seed đích

Ba con số to nhất, không mảnh nào nối X tới D.

Và một tập cạnh vẫn không nói thứ tự — mà thứ tự chính là thứ multi-hop cần đọc ra.

Ràng buộc thứ tự là nhân quả, không phải quy ước

tầng 1 Phim-X → Ông-Z Ông-Z mới «biết» nó liên quan truy vấn
tầng 2 Ông-Z → D tin này mới có nội dung

Nếu tầng 1 không gửi thì sang tầng 2 Ông-Z vẫn rỗng — tin nó gửi cho D chẳng mang gì.

Cạnh điểm IG cao mà nguồn chưa nhận được gì thì không truyền được gì.

Điểm chốt

Cần một chuỗi liền mạch theo thời gian: cạnh thứ i dùng ở đúng tầng i. Đó chính là một circuit.

Chi tiết: §4.6

Tầng B · ba bước: đo → nối → thử

1 · Đo — IG trên gate

Vặn mọi gate theo một tham số α từ 0 lên 1, cộng dồn gradient.

Một lượt backward trả về đạo hàm theo mọi gate cùng lúc.

M ≈ 20 lượt, bất kể bao nhiêu cạnh.

2 · Nối — vẽ lại rồi quét

Vẽ mỗi node một lần cho mỗi tầng. Mọi cạnh đi sang phải đúng một cột → DAG, không chu trình.

Quét một lượt trái sang phải, mỗi ô giữ lối tốt nhất tới nó.

3 · Thử — suff

Bật đúng đường đó, tắt hết còn lại, chạy model thật.

suff = phần cái bánh lấy lại được. Nhận khi suff ≥ 0.95.

1 lượt forward mỗi đường.

Bảng IG của một chuỗi sạch — chú ý hình chéo

cạnh	tầng 1	tầng 2	tầng 3	tầng 4
X → C	0.11	0.01	0.00	0.00
C → D_7	0.00	0.14	0.01	0.00
D_7 → B	0.00	0.00	0.17	0.01
B → D_{15}	0.00	0.00	0.00	0.21

Điểm chốt

IG là bộ sàng, không phải bằng chứng — nó nói «vặn mạnh thì tăng», không nói «chừng này là đủ».

Chi tiết: §4.4, §4.6 – §4.10, Kiểm tra 4 và 5

Tầng B · đồ thị trải phẳng và quy hoạch động

Vẽ mỗi node một lần cho mỗi tầng

```
V^  =  V  ×  { 0, 1, ..., L }

E^  =  { ((v',l-1),(v,l)) }   cạnh thật,
                               trọng số IG_e^(l)
    ∪  { ((v,l-1),(v,l))  }   đứng yên, trọng số 0
```

Ba thứ rơi ra ngay

```
1  ràng buộc tầng biến mất -- đã nằm trong hình
2  không có chu trình -> quét một lượt theo cột
3  đích là o (D, L), không phải (D, bất kỳ)
   vi p(D) tinh tu h_D^(L)
   (thêm dòng cho cân bằng)
```

Quy hoạch động

```
best[(v,0)] = 0     nếu v thuộc V_q
            = -inf  nếu không

best[(v,l)] = max {
   best[(v, l-1)]                  đứng yên, +0
   best[(v',l-1)] + IG_e^(l)       mọi e=(v',r,v)
}
back[(v,l)] = cách nào thắng  (để truy ngược)
```

Chi phí: $O((|V| + |\mathcal{E}|) \times L)$ — một lượt quét, không gọi model.

σ(D) — miễn phí cùng lượt quét

```
reach[(v,0)] = {s}  nếu v = s thuộc V_q

reach[(v,l)] = reach[(v, l-1)]
               ∪  hợp reach[(v', l-1)]

sigma(D) = | reach[(D,L)] |

best lấy max, reach lấy hợp
```

Điểm chốt

$\sigma(D) = 0 \iff \text{best}[(D,L)] = -\infty$. Hai tiền kiểm của §6.1 là cùng một phép thử.

Chi tiết: §4.7, §4.8

Tầng B · IG path trên bài toán này — chạy tay

X --e1--> C --e2--> D_7 --e3--> B --e4--> D_{15} **V_q = { X }, đích D_{15}, L = 4**

1 · IG chấm từng (cạnh, tầng) — không phải từng cạnh

cạnh	tầng1	tầng2	tầng3	tầng4
e1 X→C	0.11	0.01	0.00	0.00
e2 C→D_7	0.00	0.14	0.01	0.00
e3 D_7→B	0.00	0.00	0.17	0.01
e4 B→D_{15}	0.00	0.00	0.00	0.21

Hình chéo = một chuỗi sạch. Chỉ số tầng đọc ra luôn thứ tự hop.

2 · Quét trái → phải, mỗi ô giữ lối tốt nhất

best[(v,l)]	t0	t1	t2	t3	t4
X	0	0	0	0	0
C	-inf	0.11	0.11	0.11	0.11
D_7	-inf	-inf	0.25	0.25	0.25
B	-inf	-inf	-inf	0.42	0.42
D_{15}	-inf	-inf	-inf	-inf	0.63

Ranh giới -inf chạy chéo = mặt sóng thông tin.

3 · Truy ngược từ ô đích (D_{15}, tầng 4)

(D_{15},4) ←e4@t4← (B,3) ←e3@t3← (D_7,2) ←e2@t2← (C,1) ←e1@t1← (X,0)

0.11 + 0.14 + 0.17 + 0.21 = 0.63 khớp đúng giá trị ô đích ✓ rồi bật riêng đường này, chạy model, đo suff

Điểm chốt

IG chỉ cho ra một đống điểm. Chính chỉ số tầng mới biến đống điểm đó thành một đường có thứ tự.

Chi tiết: §4.6 – §4.9

Tầng B · vì sao cộng chứ không nhân

Phản xạ thông thường: đặt cost = −log(w) rồi tìm đường rẻ nhất, như Dijkstra trên đồ thị xác suất.
Ở đây làm vậy là sai loại — IG là phân rã cộng tính, −log rồi cộng thì tương đương nhân.

Hai đường giá trị hoàn toàn bằng nhau, c = 0.60

		max-sum	min(−log)
A	2 hop	0.30 + 0.30 = 0.60	2 × 1.20 = 2.41
B	6 hop	6 × 0.10 = 0.60	6 × 2.30 = 13.82

max-sum: hòa — đúng, chúng đáng giá như nhau.

min(−log): A thắng gấp 5.7 lần — sai.

Phạt hai lần, tách ra bằng công thức

chi phí(h) = h·(−log c) + h·log h

- h·(−log c): do số cạnh tuyến tính
- h·log h: do c/h pha loãng trên tuyến tính

Vì sao

Đường dài chính là thứ ta muốn tìm — multi-hop là lý do tồn tại của cả tầng B.

Hàm mục tiêu thiên vị đường ngắn sẽ chọn lối tắt giả 2 hop và bỏ chuỗi suy luận thật 4 hop.

Chi tiết: §4.9

Tầng B · kiểm chứng suff / nec, và trần n_max

Che gate theo Π rồi chạy một lượt

```
g_Pi[e,l] = g*[e,l]  nếu (e,l) thuộc Pi,   0 ngược lại

suff(Pi) = [ F_B(g_Pi) - F_B(0) ] / [ F_B(g*) - F_B(0) ]     phần cái bánh Pi lấy lại được
nec(Pi)  = [ F_B(g*) - F_B(g_!Pi) ] / [ F_B(g*) - F_B(0) ]   phần mat khi tắt Pi

nhận Pi  <=>  suff(Pi) >= theta_s = 0.95
```

n_max là suy ra, không phải chọn

```
eps~ = eps / cái bánh        mặc định 0.10

mọi đường được nhận:  gain_i >= eps
cac gain rời nhau (cạnh đã lấy bị đặt IG=0)
vòng lặp còn chạy: tích lũy < theta_s * cake

(n-1)*eps <= tổng gain < theta_s * cake
=>  n <= ceil( theta_s / eps~ ) = 10
```

Trần của chính suff — credit mồ côi

```
Node không với tới được từ seed vẫn gửi thông
diep, vấn xê dịch p(D) -> nằm trong mẫu số
ma không đường nao chạm tới (tu so).

E_reach  = { (e,l) : best[(v',l-1)] > -inf }
suff_max = tổng IG trên E_reach / tổng IG
suff_max < theta_s -> dừng ngay, vô ích
```

Điểm chốt

eps và n_max là một núm, không phải hai. Hằng n_max = 5 tương đương $\tilde{\varepsilon} = 0.19$ — hà khắc hơn vẻ ngoài.

Chi tiết: §4.10, §4.11

Tầng B · hội tụ nhiều seed — greedy xóa cạnh

Tiền kiểm, rồi hai pha

```
tiền kiểm   nếu suff_max < theta_s:  dừng, ghi "không giải thích được bằng đường từ seed"

-- pha 1: gom ------------------------------------------------------------
Pi_D <- rỗng
lặp:
    pi   = truy ngược back[] tu (D,L)
    gain = tổng IG của cac cạnh mới trong pi
    Pi_D <- Pi_D + pi
    nếu suff(Pi_D) >= theta_s   ->  dừng:  thành công
    nếu gain < eps              ->  dừng:  bằng chứng phân tán
    nếu |Pi_D| = n_max          ->  dừng:  hết ngân sách đọc hiểu
    đặt IG_e^(l) = 0 cho mọi (e,l) thuộc Pi_D      # đã trả tiền rồi
    chạy lại QHĐ

-- pha 2: tỉa ngược ------------------------------------------------------
cho mỗi pi trong Pi_D (theo thứ tự da them):
    nếu suff(Pi_D \ {pi}) >= theta_s:   Pi_D <- Pi_D \ {pi}     # đường thừa, bỏ
```

Vì sao xóa cạnh

```
cạnh mới   vẫn có giá trị -> đường kế tiếp
           được thưởng vì mang thêm cái mới
cạnh đã có giá trị 0 -> dùng lại miễn phí
```

Ba lý do dừng = ba chẩn đoán

suff đạt	thành công
gain < eps	bằng chứng phân tán (đọc kèm sigma)
|Pi| = n_max	đồ thị dư thừa cao -- không phải lỗi

Chi tiết: §4.11

Nối hai tầng — chỉ một thứ đi qua ranh giới



Hợp lệ ✓

Nối một tập.

T là đầu ra của tầng A và là đầu vào (đích) của tầng B. Một tập hợp đi qua ranh giới thì không mang theo đơn vị đo nào.

Vô nghĩa ✗

Nhân hai điểm số.

F và F_B có mục tiêu khác nhau và baseline khác nhau, nên chuỗi tính đầy đủ không bắc cầu. Không có bất biến nào nối tầng A với tầng B.

Điểm chốt

Nối tập, đừng nhân điểm. Trên giao diện thì hiển thị hai cột riêng.

Chi tiết: §6, phụ lục §1

Dùng hai tầng làm tín hiệu train retriever

L_{node} ← T

«node nào đúng»

Thay nhãn cũ «đoạn có chứa chuỗi đáp án» — nhãn cũ đầy dương tính giả.

Không đổi kiến trúc.

L_{hop} ← Π

«đi tới đó bằng lối nào»

Giám sát trung gian từng tầng, vá chỗ credit assignment bị đứt qua L bước.

Thêm một head nhỏ mỗi tầng, và bắt buộc lấy mẫu âm.

L_{edge} ← IG cạnh

«cạnh nào nên mở»

Dạy trực tiếp cho gate phụ thuộc truy vấn.

Cần module gate — đổi kiến trúc, làm sau cùng.

Cả ba loss đọc từ một lượt forward, nên chi phí train không nhân ba. Cả ba cũng cập nhật chung một mạng.

Thứ tự bị ép bởi phụ thuộc dữ liệu

```
L_node   cần T   có sẵn, không phụ thuộc model
L_hop    cần Π   Π đào bằng IG trên model hiện tại
L_edge   cần ĝ   cần kiến trúc gate

Nên: train L_node tới hội tụ → chẩn đoán → mới thêm.
```

Hai chỗ dễ sai

Bộ lọc chỉ áp L_{hop} / L_{edge} khi model đang xếp sai và đường đã kiểm — điều kiện tự co lại khi model khá lên, không cần lịch giảm λ.

Miền cộng L_{hop} cộng hết thì dương/âm ≈ 10 trên 10^7. Phải lấy mẫu âm; âm khó nhất là đúng node, sai tầng.

Chi tiết: §6.2 – §6.4

Ba hàm mất mát — công thức đầy đủ

$L = L_{node} + \lambda_1 \cdot L_{hop} + \lambda_2 \cdot L_{edge}$

$L_{node} = -\sum_{D \text{ thuộc } T} w(D) * \log p(D) \; - \; \sum_{v \text{ thuộc } V_{neg}} \log(1 - p(v))$

$L_{hop} = \sum_{l=1..L} \; \sum_{v \text{ thuộc } V_{hop}(l)} \; BCE(p^{(l)}(v), \; 1[(v,l) \text{ thuộc } Pi])$

$L_{edge} = \sum_{(e,l)} \; BCE(g^{}_e^{(l)}, \; w^{}_e^{(l)})$ $\qquad$ $w^{} = IG$ chuẩn hóa ve $[0,1]$

Cả ba đọc từ một lượt forward -> chi phí train không nhân ba.

$V_{hop}(l)$ — bắt buộc lấy mẫu âm

$V_{hop}(l) = \{ v : (v,l) \text{ thuộc } Pi \}$ $\qquad$ dương

$\cup \{ v : (v,l') \text{ thuộc } Pi, \; l' \neq l \}$ $\qquad$ âm khó

$\cup \; V_{neg}$ $\qquad$ âm thường

Cộng trên mọi (v,l): đường/am ~ 10 : 10^7

-> "luôn đoán 0" đã gần tối ưu, loss đứng yên

Bộ lọc, và ngưỡng theo từng ví dụ

Chỉ áp L_{hop} / L_{edge} khi thỏa cả ba:

- D thuộc T
- $rank_{GFM}(D) > k_{top}(q)$ — model đang xếp sai
- $suff(Pi_D) \geq theta_s$ — đường đã kiểm chứng

$k_{top}(q) = ceil(beta \times |T(q)|)$, $\quad beta \sim 1.5$

Điểm chốt

Âm khó — đúng node, sai tầng — mới là nhóm dạy được thời điểm, thứ nhãn ở tầng cuối mù hoàn toàn.

Chi tiết: §6.3, §6.4

Bảng chẩn đoán — và nó cũng chính là nguồn nhãn

Hai đại lượng, đo hai thứ hoàn toàn độc lập nhau — nên xếp thành hai trục, không phải một danh sách.

	rank ≤ k_top(q) **model lấy được D**	**rank > k_top(q)** **model bỏ sót D**
suff ≥ θ_s **có lối, đã kiểm**	ổn — không cần làm gì	**lỗi model → tập S_k** **đường tồn tại mà model không đi; Π, ŵ là nhãn train**
suff < θ_s **không lối nào đủ**	**model đúng mà không lối nào giải thích nổi**	lỗi đồ thị bổ sung cạnh lúc xây graph

Nó chạy ở mỗi mốc, không phải một lần

```
mốc 0   đào T → train L_node tới hội tụ
mốc 1   ▸ bảng chẩn đoán ◂ → ô «lỗi model» = S₁
              → train L_node + λ₁·L_hop
mốc 2   ▸ bảng chẩn đoán ◂ → thêm gate, L_edge
```

Một lần chạy, hai cách đọc

báo cáo	phân bố các ô = tiến độ của đợt train vừa xong
nguồn nhãn	ô «lỗi model» chính là tập ví dụ để đào Π và ŵ cho đợt tới

Điểm chốt

Chẩn đoán và đào nhãn là cùng một thao tác — chỉ khác cách đọc kết quả.

Chi tiết: §6.1, §6.5

Chi phí — và vì sao phải hai pha

Một truy vấn, pipeline đầy đủ

```
cost(S) ≈ 500 + 250·|S| — các lượt KHÔNG bằng giá
vòng 0  v(∅)+v(R)+v(C)+40 singleton    44.500
vòng 1  (r−1)·m = 24 lượt              27.000
vòng 2  ~30 lượt                       37.000
vòng 3  1 + 6 lượt                      6.500
                                      ────────
                                       115.000  ≈ $0,35
```

Quy mô thật — 200.000 truy vấn

```
pha 1  18.200 nạp → 10.000 chạy =   $3.800
pha 2 200.000 nạp → 110.000     =   $7.500
cứu     3.000 ca × $0,34        =   $1.000
                                  ─────────
hiệu chuẩn 500 HotpotQA           =     $185
                                  $12.485   ~104k nhãn ĐÁNG
TIN
chạy pipeline đầy đủ cả 200k   $76.000
6 lần rẻ hơn — nhờ bỏ vòng 0 và cửa thoát C0
```

v(C) nặng nhất, và đó là đúng giá

```
C0  chẻ + bảng phủ            400 token    2%
C1  v(∅)                      500 token    2%
C3  v(R)                    3.500 token   16%
C4  v(C)                   10.500 token   47%
C6  verification            7.400 token   33%
   22.300 chạy đủ · 12.400 kỳ vọng (45% thoát)
Bù v(C) bằng hằng số → ~10% truy vấn bị trần thấp
→ nhãn sai mà verification KHÔNG bắt được.
```

Sổ dữ liệu — 2,61 triệu mẫu

```
pha 1, bốn vòng      10.000 ×111 = 1.110.000
verification pha 2  110.000 × 10 = 1.100.000
cứu                   3.000 ×111 =   333.000
trượt bị bỏ          14.000 ×  5 =    70.000
thoát ở C0           98.200 ×  0 =         0
                                   ──────────
                                   2.613.000
```

Bảng này chưa được kiểm chứng — nói thẳng

p = 0,75 là GIẢ ĐỊNH, chưa đo, mà toàn bộ cột chi phí treo vào nó. «30% mỗi lần thử lại» và «~4% có |S*| ≥ 6» cũng là giả định. Cổng B3 tồn tại chính vì ba con số đó — và nó miễn phí, nên phải chạy TRƯỚC.

Chi tiết: §7

Phạm vi đóng góp và những gì chưa chắc

Từ literature

Deegan–Packel · Public Good Index
Banzhaf (ở đây dùng bản có dấu)
Integrated Gradients
Mở rộng đa tuyến tính → tính chất c/h
PDist / LOOP của Atlas (JMLR 2023)
GFM-RAG, G-reasoner (NeurIPS 2025 / 2025)
Mask cạnh để giải thích GNN

Đề xuất trong tài liệu này

Surrogate ở biên + beam search thay cho 2^K
Banzhaf có dấu để bắt đoạn đẩy ngược
IG gắn vào gate cạnh thay vì embedding
Gate riêng từng tầng, và gate phụ thuộc truy vấn
max-sum trên DAG trải phẳng (thay min-cost)
Greedy xóa cạnh + tỉa ngược cho hội tụ
Tập đủ tối tiểu làm nhãn train retriever

Chi tiết: §1 «Phạm vi đóng góp», §3.10, §8, §6.7